

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Техническое зрение»
Тема: Предобработка изображений

Студент гр. 2491

Недовизий А.И.

Преподаватель

Федоркова А.О.

Санкт-Петербург

2025

Цель работы: изучить основные операции с изображениями, методы фильтрации изображений, пороговые преобразования.

Задания к работе:

Задание 1.

Для выполнения этого задания используйте любое изображение. Напишите программу, которая будет выводить изображение на экран следующим образом: 1. в цвете в полном размере на 5 секунд, затем закрыть; 2. В оттенках серого в полном размере на 7 секунд, затем закрыть; 3. в цвете в 2 раза меньше, чем исходный размер, на 9 секунд, затем закрыть; 4. в оттенках серого в 4 раза меньше, чем исходный размер, на 11 секунд, затем закрыть. 5. В цвете в полном размере, поменяв местами зелёный и красный каналы на 4 секунды. 6. Закрыть при нажатии на клавишу Esc. Все действия должны выполняться в одном скрипте.

Задание 2.

Используя пороговые преобразования, добиться того, чтобы фигура девушки попала в одну группу, а фон – в другую. (изображение 1-1) Какая функция для этого подошла лучше всего? Ручной поиск, адаптивный или автоматический?

Задание 3.

На изображении 1-2 выделить разметку.

Задание 4.

Используя пороговые преобразования, добиться того, чтобы горы попали в одну группу, а фон – в другую. (изображение 1-3) Какая функция для этого подошла лучше всего? Ручной поиск, адаптивный или автоматический?

Задание 5.

Для выполнения этого задания используйте изображение 1-4. Примените к нему все известные вам фильтры для сглаживания изображений. Определите, какие из них больше всего подходят для того, чтобы максимально убрать все неровности на изображениях.

Задание 6.

Отфильтруйте изображение 1-5 так, чтобы можно было распознать содержимое изображения.

Ход выполнения работы:

Задание 1: будем использовать первое изображение. Код программы приведен в листинге 1.

Листинг 1

```
import cv2
import numpy as np

# for mac
# path1 = "/Users/new/Desktop/course4labs/cv/lab1/1_1.jpg"
# for win
path1 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_1.jpg"

class Picture:
    def __init__(self, path, readmode, lifetime, winName) -> None:
        self.path = path
        self.readmode = readmode
        self.lifetime = lifetime
        self.win_name = winName

    def drawAndClosePic(self):
        image = cv2.imread(self.path, self.readmode)
        cv2.namedWindow(self.win_name, cv2.WINDOW_FULLSCREEN)
        cv2.moveWindow(self.win_name, 0, 0)
        cv2.imshow(self.win_name, image)
        cv2.waitKey(delay=self.lifetime)
        cv2.destroyWindow(self.win_name)
```

```

def drawAndWaitKey(self):
    image = cv2.imread(self.path, self.readmode)
    cv2.namedWindow(self.win_name, cv2.WINDOW_FULLSCREEN)
    cv2.moveWindow(self.win_name, 0, 0)
    cv2.imshow(self.win_name, image)
    key = cv2.waitKey(0)
    while key != 27:
        key = cv2.waitKey(0)
    cv2.destroyAllWindows()

```

```

def swapChannels(self):
    image = cv2.imread(self.path, cv2.IMREAD_COLOR)
    b, g, r = cv2.split(image)
    img_rgb = cv2.merge([b, r, g]) # type: ignore
    cv2.namedWindow(self.win_name, cv2.WINDOW_FULLSCREEN)
    cv2.moveWindow(self.win_name, 0, 0)
    cv2.imshow(self.win_name, img_rgb)
    cv2.waitKey(delay=self.lifetime)
    cv2.destroyWindow(self.win_name)

```

```

image1 = Picture(
    path=path1, readmode=cv2.IMREAD_COLOR, lifetime=5000,
winName="fullcolor"
)
image2 = Picture(
    path=path1, readmode=cv2.IMREAD_GRAYSCALE, lifetime=7000,
winName="fullgray"
)
image3 = Picture(

```

```

        path=path1,
        readmode=cv2.IMREAD_REDUCED_COLOR_2,
        lifetime=9000,
        winName="halfcolored",
    )
    image4 = Picture(
        path=path1,
        readmode=cv2.IMREAD_REDUCED_GRAYSCALE_4,
        lifetime=11000,
        winName="quartergrey",
    )
    l = [image1, image2, image3, image4]
    for item in l:
        item.drawAndClosePic()

    image5 = Picture(
        path=path1, readmode=cv2.IMREAD_COLOR, lifetime=4000,
winName="fullcolor"
    )
    image5.swapChannels()
    image5.win_name = "press esc to exit"
    image5.drawAndWaitKey()

```

На рисунках 1, 2 изображены изображения, полученные в результате выполнения программы. На рисунке 1 картинка была считана в изначальных цветах, а на рисунке 2 картинка была считана в оттенках серого. На рисунке 3 представлена картинка, в которой зеленый и красный каналы поменяны местами.

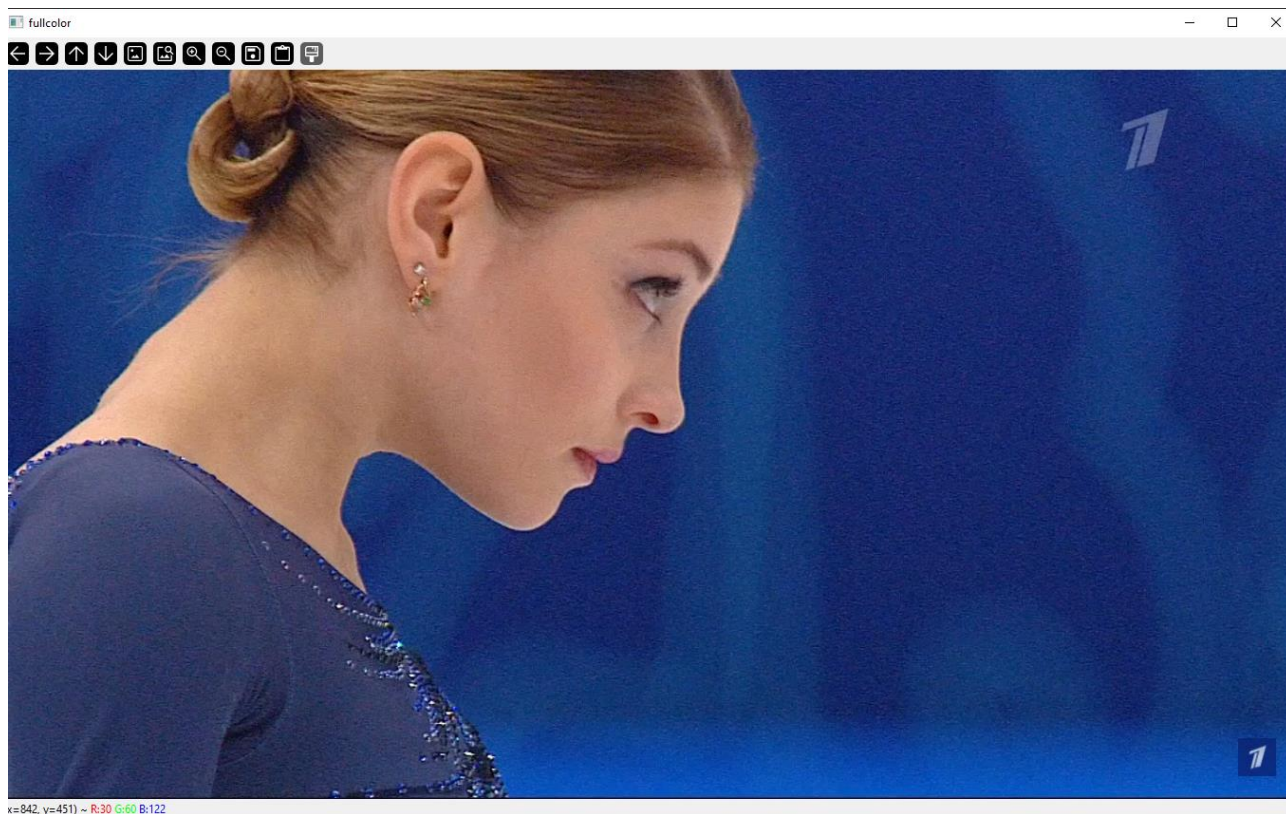


Рисунок 1 – Картинка в изначальном цвете

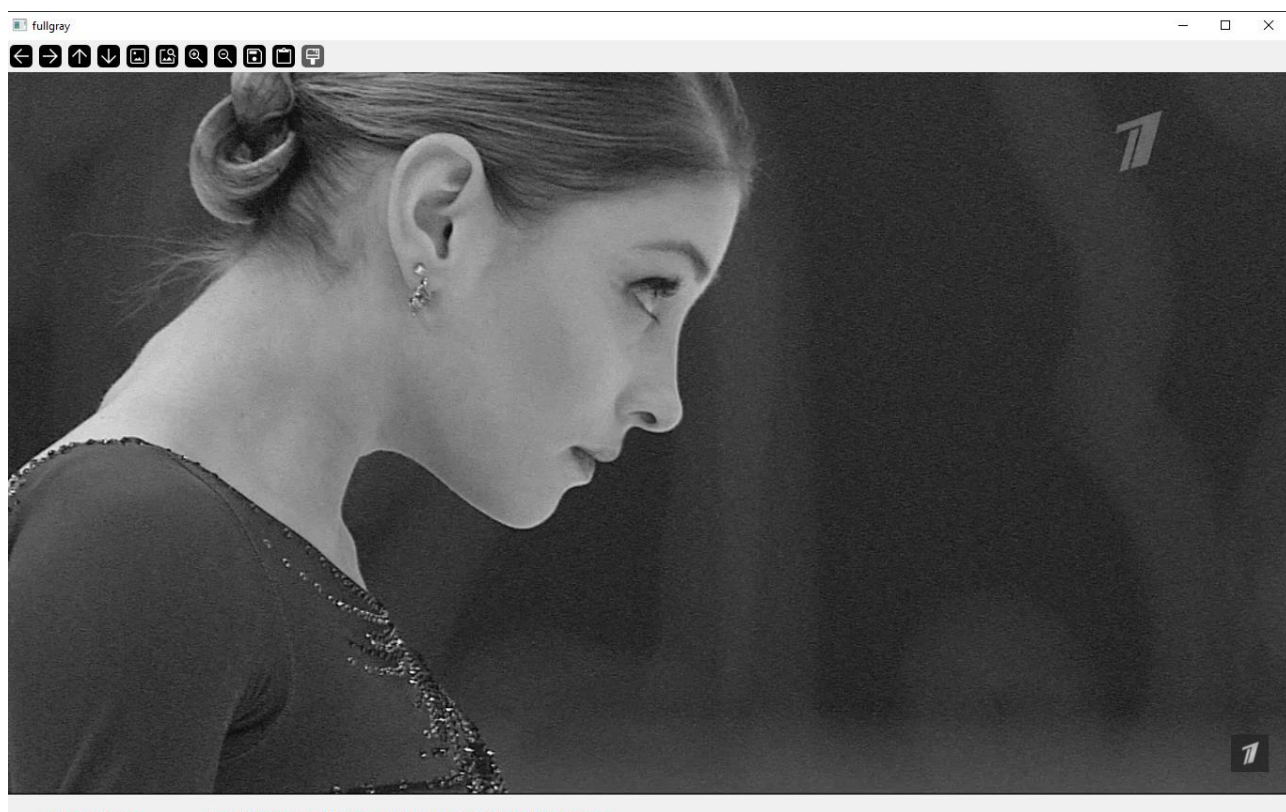


Рисунок 2 – Картинка в серых оттенках

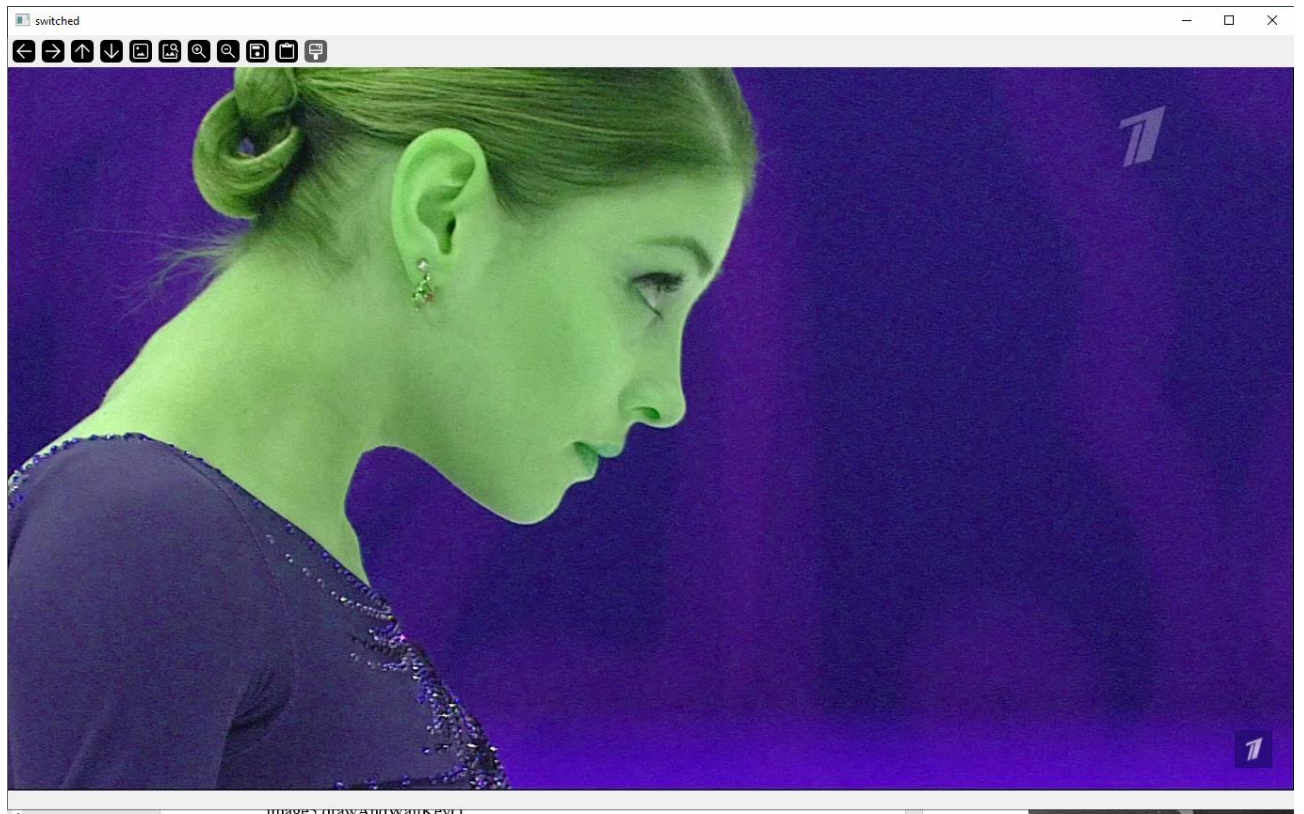


Рисунок 3 – Картинка с поменянными красным и зелеными каналами

Задание 2: Код программы приведен в листинге 2

Листинг 2:

```
import cv2
```

```
import numpy as np
```

```
path1 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_1.jpg"
```

```
# Загрузка изображения в градациях серого
```

```
image = cv2.imread(path1, cv2.IMREAD_GRAYSCALE)
```

```
# Определение порога
```

```
threshold_value = 88
```

```
# Ручной метод
```

```
# Пороговое преобразование до нуля
_, tozero = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_TOZERO)
```

```
# Отображение результатов
cv2.namedWindow("Original", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Original", 0, 0)
cv2.imshow("Original", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.namedWindow("Threshold", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Threshold", 0, 0)
cv2.imshow("Threshold", tozero)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
# Автоматический метод подбора порогового значения
# метод Отсу
tr1, otsu = cv2.threshold(image, threshold_value, 255, cv2.THRESH_OTSU)

# Метод треугольника
tr2, triangle = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_TRIANGLE)

cv2.namedWindow("Original", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Original", 0, 0)
cv2.imshow("Original", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.namedWindow("otsu", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("otsu", 0, 0)
cv2.imshow("otsu", otsu)
```

```

cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.namedWindow("triangle", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("triangle", 0, 0)
cv2.imshow("triangle", triangle)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Adaptive method
adaptive_thresh = cv2.adaptiveThreshold(
    image,          255,          cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY_INV, 11, 23
)
cv2.namedWindow("Adaptive Threshold", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Adaptive Threshold", 0, 0)
cv2.imshow("Adaptive Threshold", adaptive_thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

При исполнении этой программы показываются изображения, измененные с помощью порогового преобразования.

На рисунке 4 изображена картинка в оттенках серого.

На рисунке 5 изображена картинка с пороговым преобразованием до нуля.

На рисунке 6 изображена картинка с пороговым преобразованием с автоматическим подбором порогового значения методом Оцу.

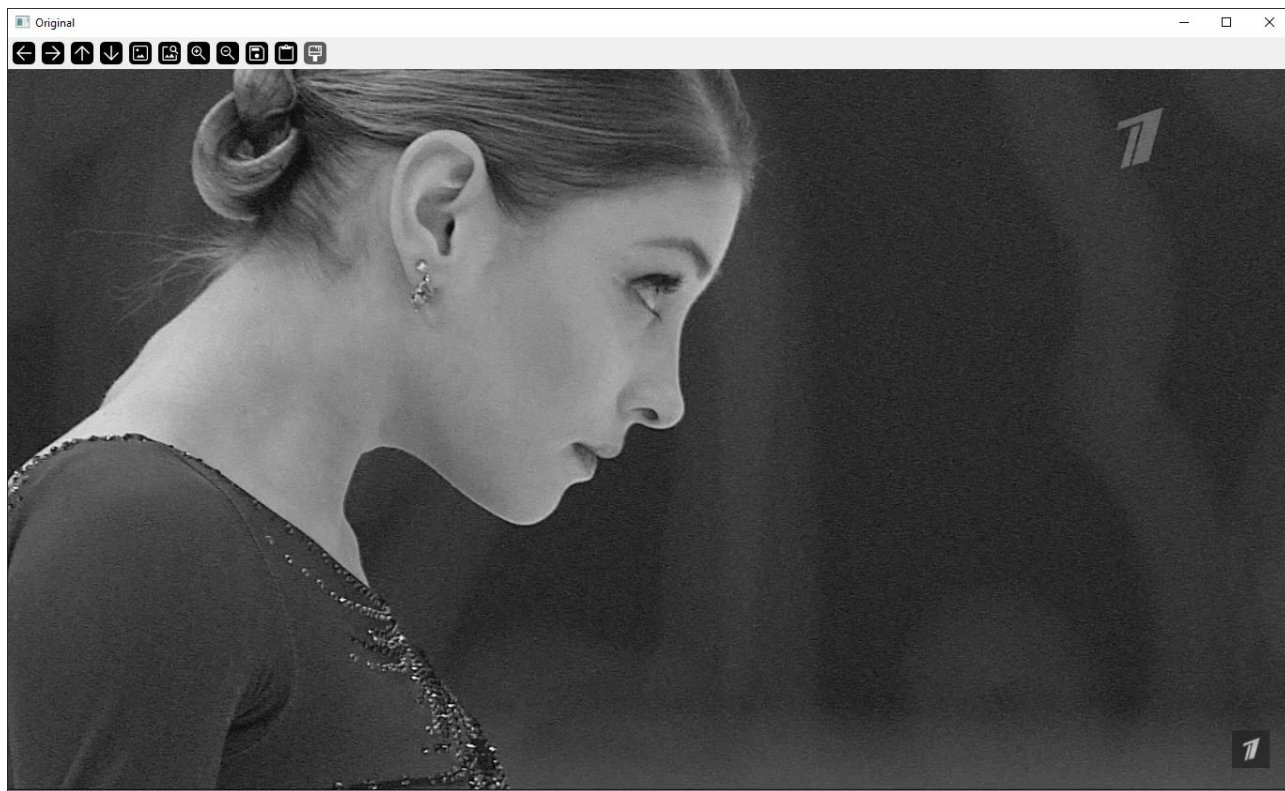


Рисунок 4 – Картинка в оттенках серого

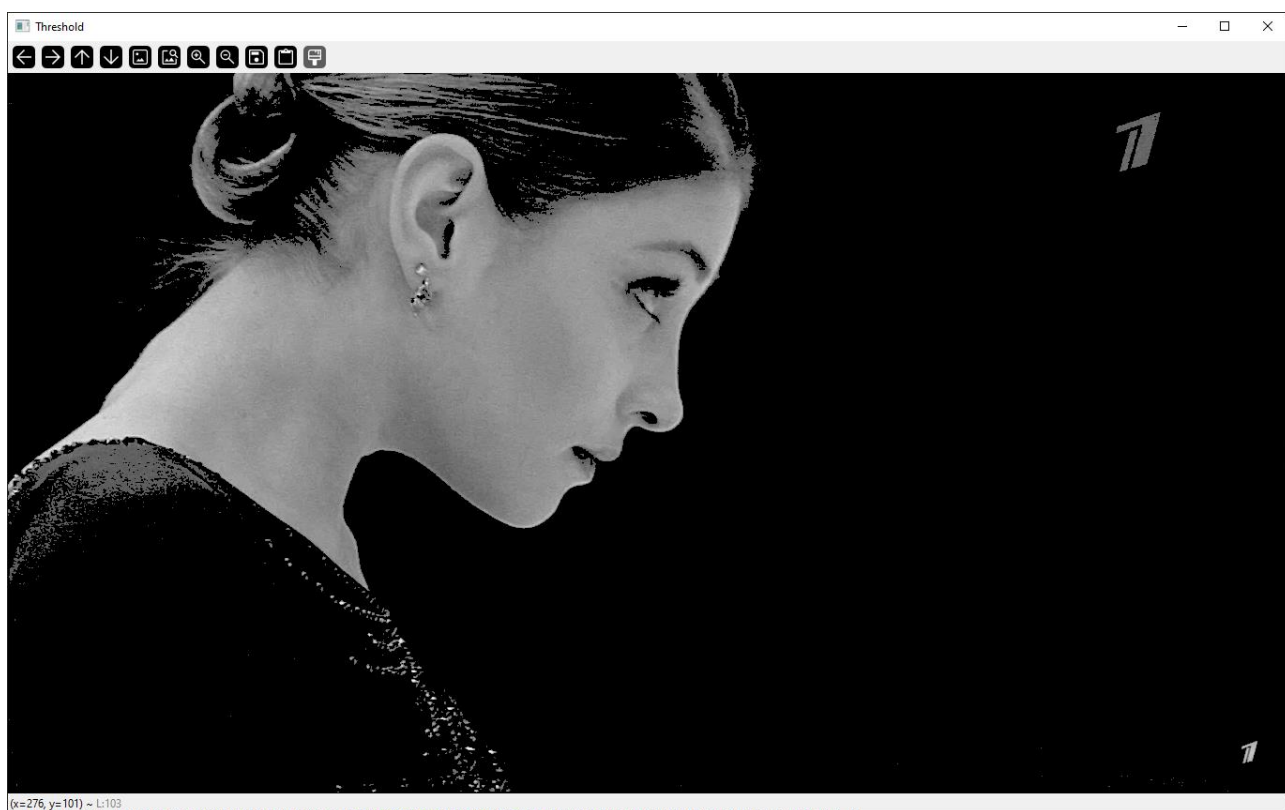


Рисунок 5 – Пороговое преобразованием до нуля

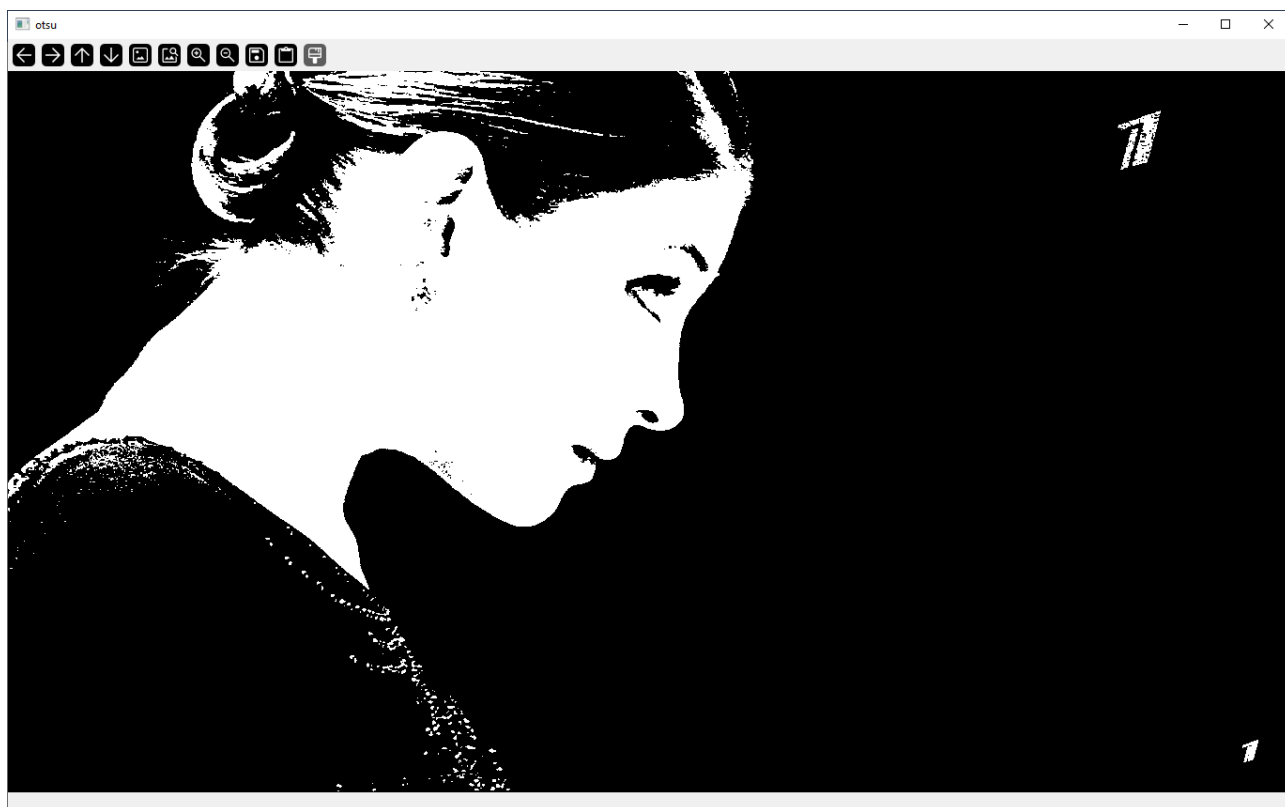


Рисунок 6 – Пороговое преобразование с применением метода Оцу для поиска порогового значения



Рисунок 7 – Пороговое преобразование с применением метода Треугольника для поиска порогового значения

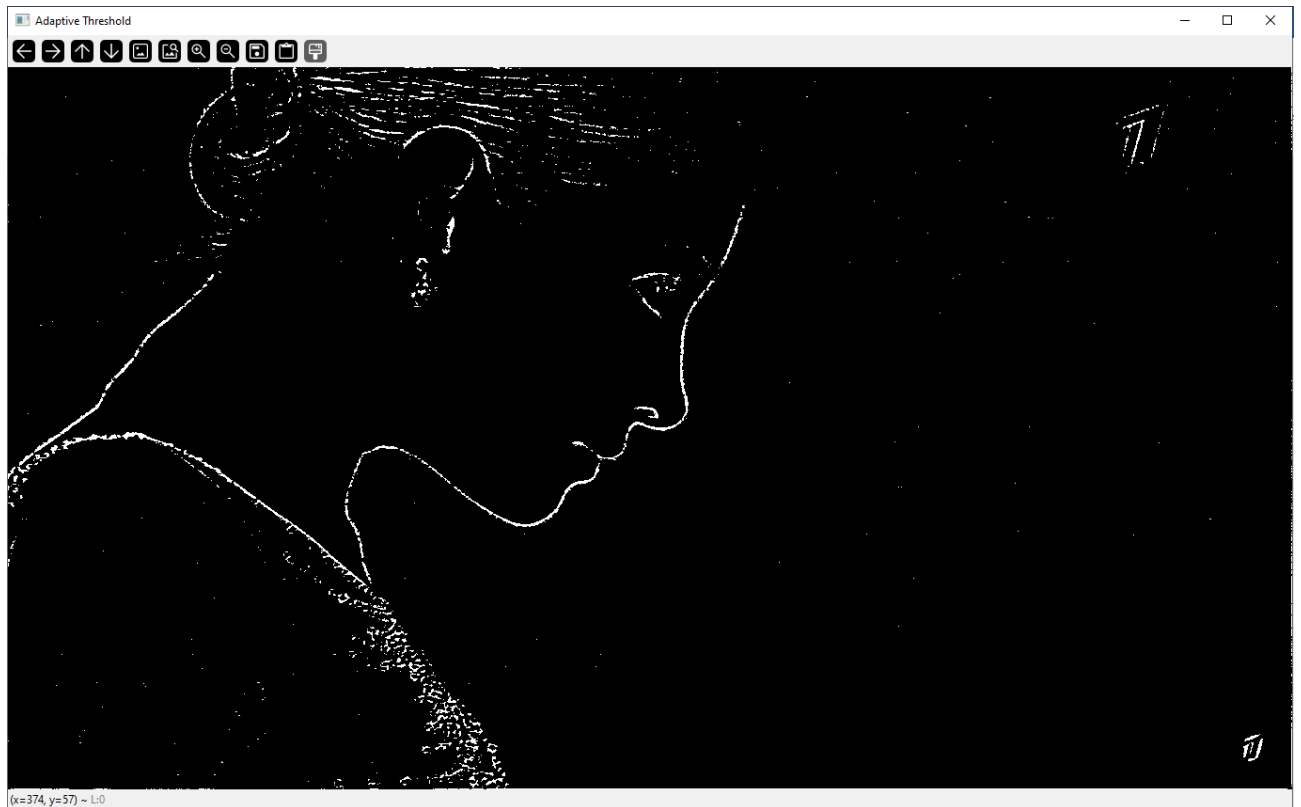


Рисунок 8 – Адаптивное пороговое преобразование

Для отделения фигуры девушки от фона хорошо подходят ручное преобразование, а также автоматическое методом Оцу и треугольника. Адаптивное выделило контур, но если мы хотим сохранить всю фигуру, то он не подойдет.

Лучшей из этих функций можно назвать пороговое преобразование с автоматическим поиском по методом треугольника, так как оно захватило даже волосы девушки.

Задание 3:

Листинг 3:

```
import cv2
import numpy as np
path2 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_2.jpg"

threshold_value = 160

image = cv2.imread(path2, cv2.IMREAD_GRAYSCALE)
```

```

cv2.namedWindow("original", cv2.WINDOW_NORMAL)
cv2.moveWindow("original", 0, 0)
cv2.imshow("original", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

_, crosswalk = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_BINARY)

cv2.namedWindow("bin", cv2.WINDOW_NORMAL)
cv2.moveWindow("bin", 0, 0)
cv2.imshow("bin", crosswalk)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

На рисунках 9 и 10 представлены изображения, сгенерированные в процессе работы программы.



Рисунок 9 – Картинка в серых оттенках



Рисунок 10 – Картинка после бинаризации

Задание 4:

```
import cv2
```

```
import numpy as np
```

```
# for mac
```

```
# path3 = "/Users/new/Desktop/code/cv/lab1/1_3.png"
```

```
# for win
```

```
path3 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_3.png"
```

```
# Загрузка изображения в градациях серого
```

```
image = cv2.imread(path3, cv2.IMREAD_GRAYSCALE)
```

```
# Определение порога
```

```
threshold_value = 120
```

```
# Ручной метод
```

```

# Пороговое преобразование до нуля
_, tozero = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_BINARY)

# Отображение результатов
cv2.namedWindow("Original", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Original", 0, 0)
cv2.imshow("Original", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.namedWindow("Threshold", cv2.WINDOW_FULLSCREEN)
cv2.moveWindow("Threshold", 0, 0)
cv2.imshow("Threshold", tozero)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Автоматический метод подбора порогового значения
# метод Отсу
tr1, otsu = cv2.threshold(image, threshold_value, 255, cv2.THRESH_OTSU)
# Метод треугольника
tr2, triangle = cv2.threshold(image, threshold_value, 255,
cv2.THRESH_TRIANGLE)
cv2.imshow("Original", image)
cv2.moveWindow("Original", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.imshow("otsu", otsu)
cv2.moveWindow("otsu", 0, 0)
cv2.waitKey(0)

```

```

cv2.destroyAllWindows()
cv2.imshow("triangle", triangle)
cv2.moveWindow("triangle", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Adaptive method
adaptive_thresh = cv2.adaptiveThreshold(
    image,          255,          cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY_INV, 11, 23
)
cv2.imshow("Adaptive Threshold", adaptive_thresh)
cv2.moveWindow("Adaptive Threshold", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

На рисунках 11-14 изображены картинки с применением разных пороговых преобразований.

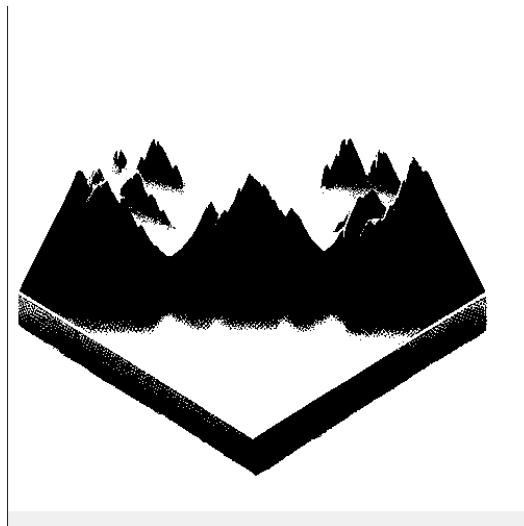


Рисунок 11 – Изображение с пороговым преобразованием с ручным порогом

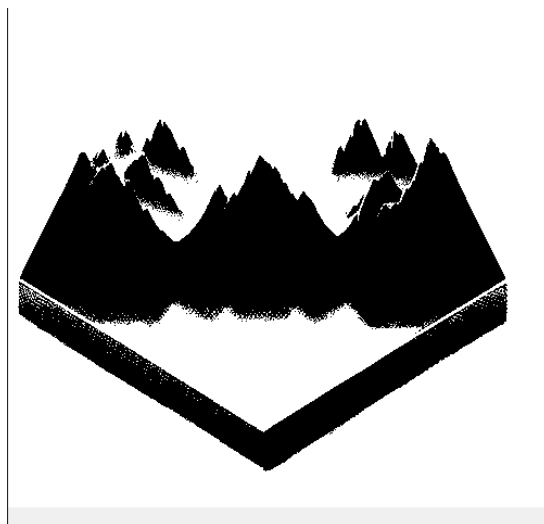


Рисунок 12 – Изображение с пороговым преобразованием с порогом, найденным по методу Оцу

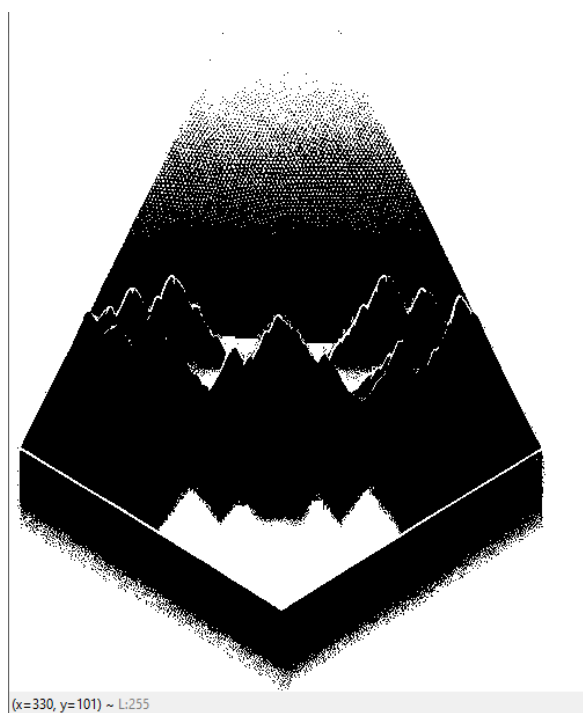


Рисунок 13 – Изображение с пороговым преобразованием с порогом найденным по методу треугольника

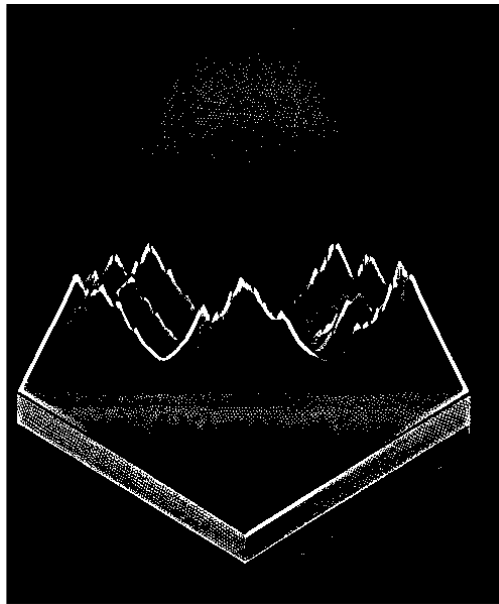


Рисунок 14 – Изображение с адаптивным пороговым преобразованием

Для поставленной задачи разделения гор и фона лучше всего подошло ручное пороговое преобразование и преобразование с порогом, найденным по методу Оцу.

Задание 5:

На рисунках 15 – 20 показаны изображения, полученные в результате выполнения программы.

Листинг 5:

```
import cv2
import numpy as np

# for mac
# path4 = "/Users/new/Desktop/code/cv/lab1/1_4.PNG"
# for win
path4 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_4.PNG"

image = cv2.imread(path4, cv2.IMREAD_COLOR)
blurredImage = cv2.blur(image, (3, 3))
```

```
boxFilteredImageN = cv2.boxFilter(image, -1, (7, 7), normalize=True)
gaussianBlurred = cv2.GaussianBlur(image, (15, 15), 0)
median = cv2.medianBlur(image, 19)
bi = cv2.bilateralFilter(image, 100, 100, 100)
```

```
cv2.imshow("original", image)
cv2.moveWindow("original", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("blurred", blurredImage)
cv2.moveWindow("blurred", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("boxedFiltered", boxFilteredImageN)
cv2.moveWindow("boxedFiltered", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("gaussian", gaussianBlurred)
cv2.moveWindow("gaussian", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("median", median)
cv2.moveWindow("median", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("bilateral", bi)
cv2.moveWindow("bilateral", 0, 0)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

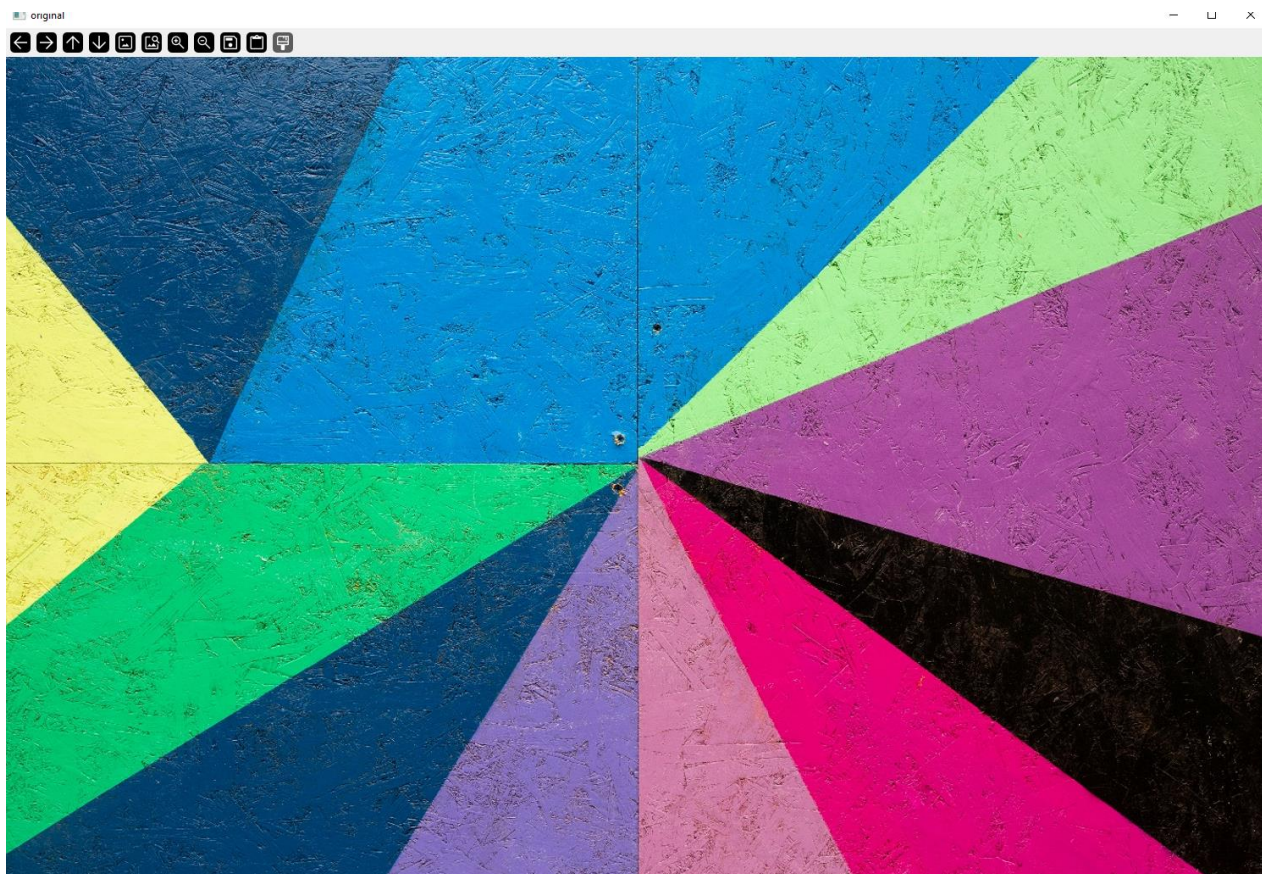


Рисунок 15 – Изображение без применения фильтров

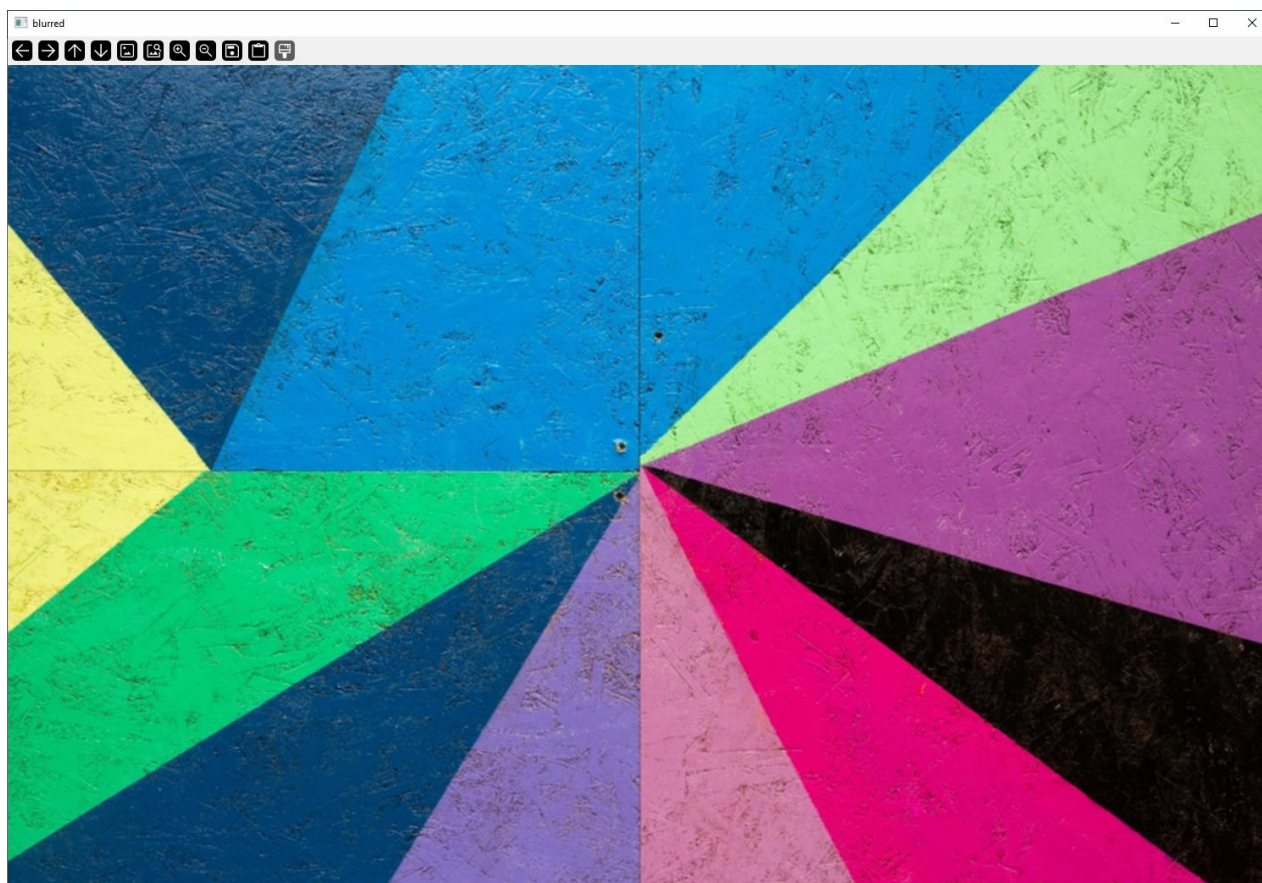


Рисунок 16 – Изображение с размытием

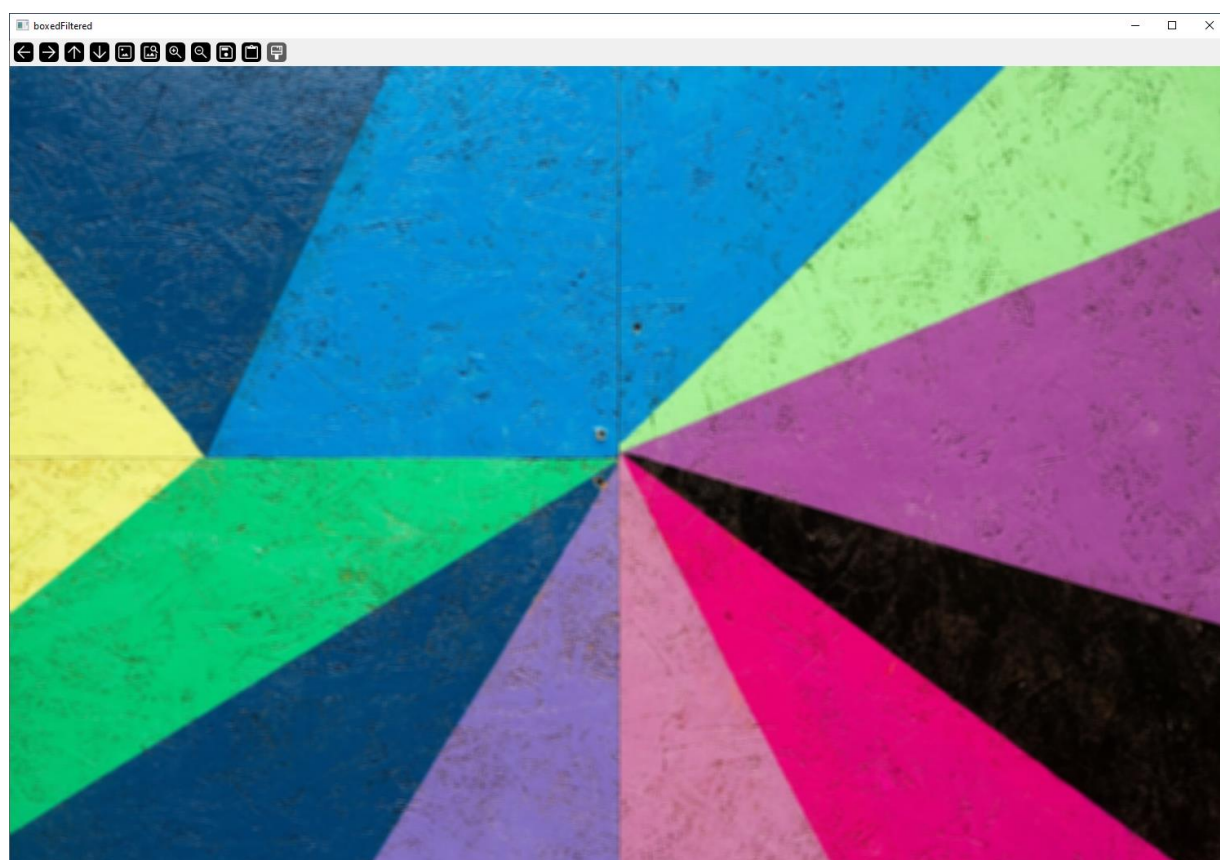


Рисунок 17 – Изображение с применением BoxFilter

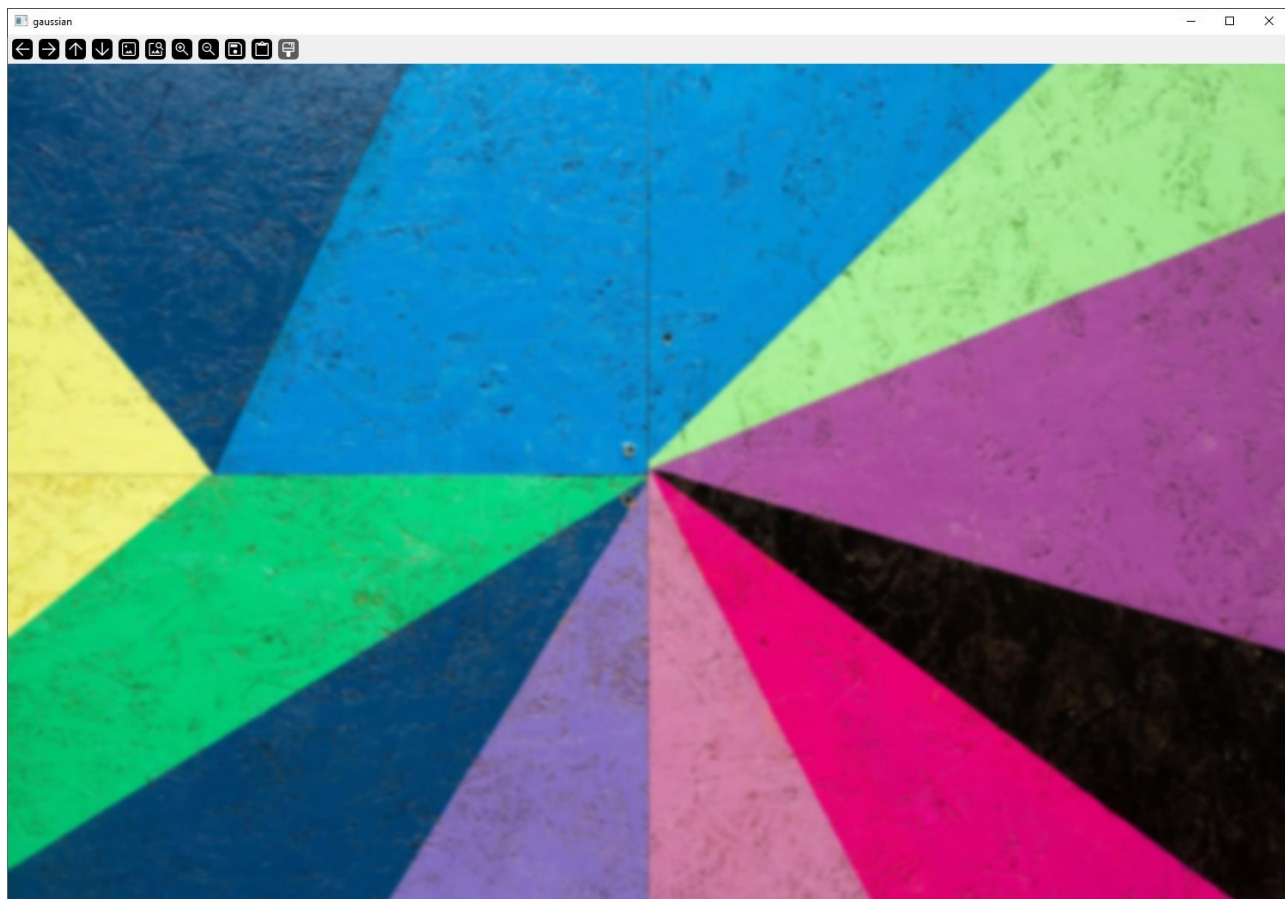


Рисунок 18 – Изображение с применением GaussianBlur

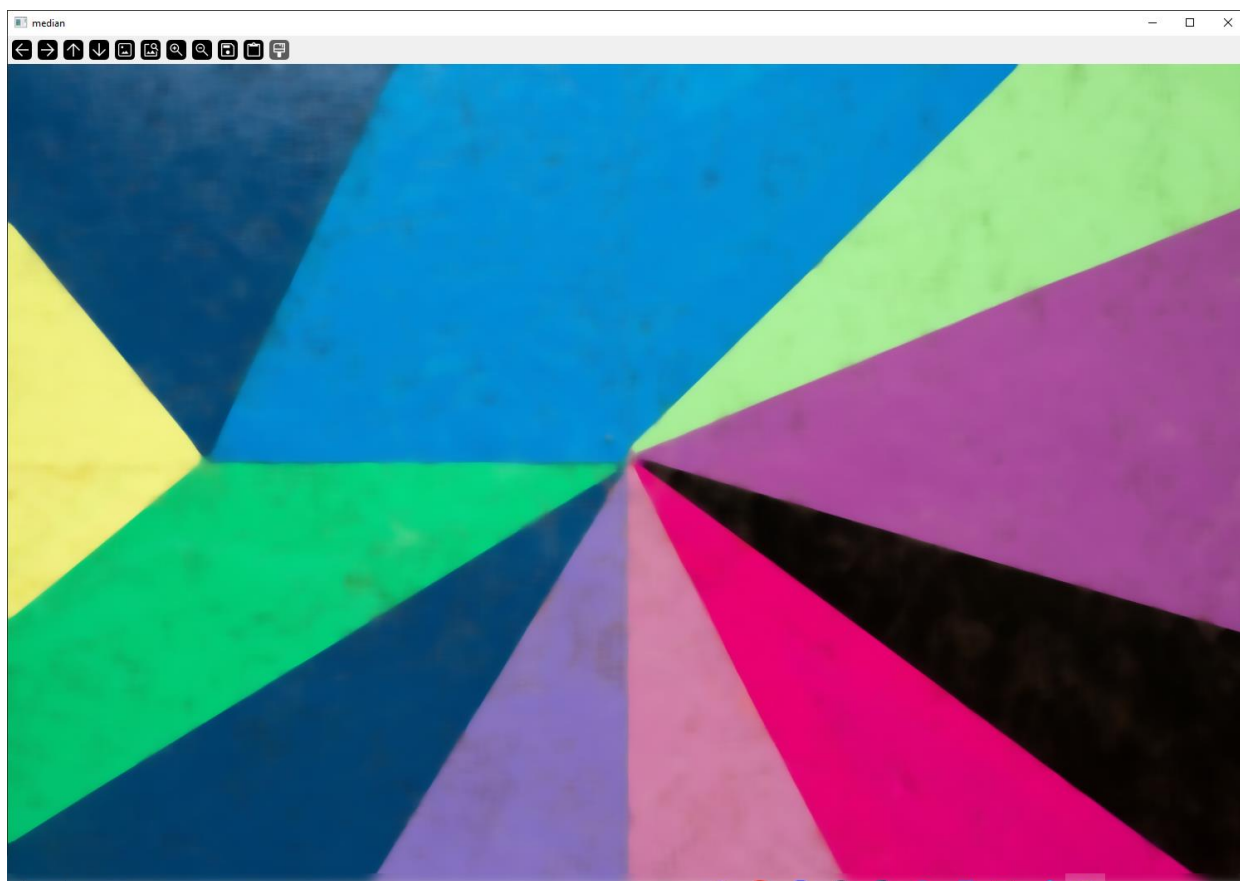


Рисунок 19 – Изображение с применением medianBlur

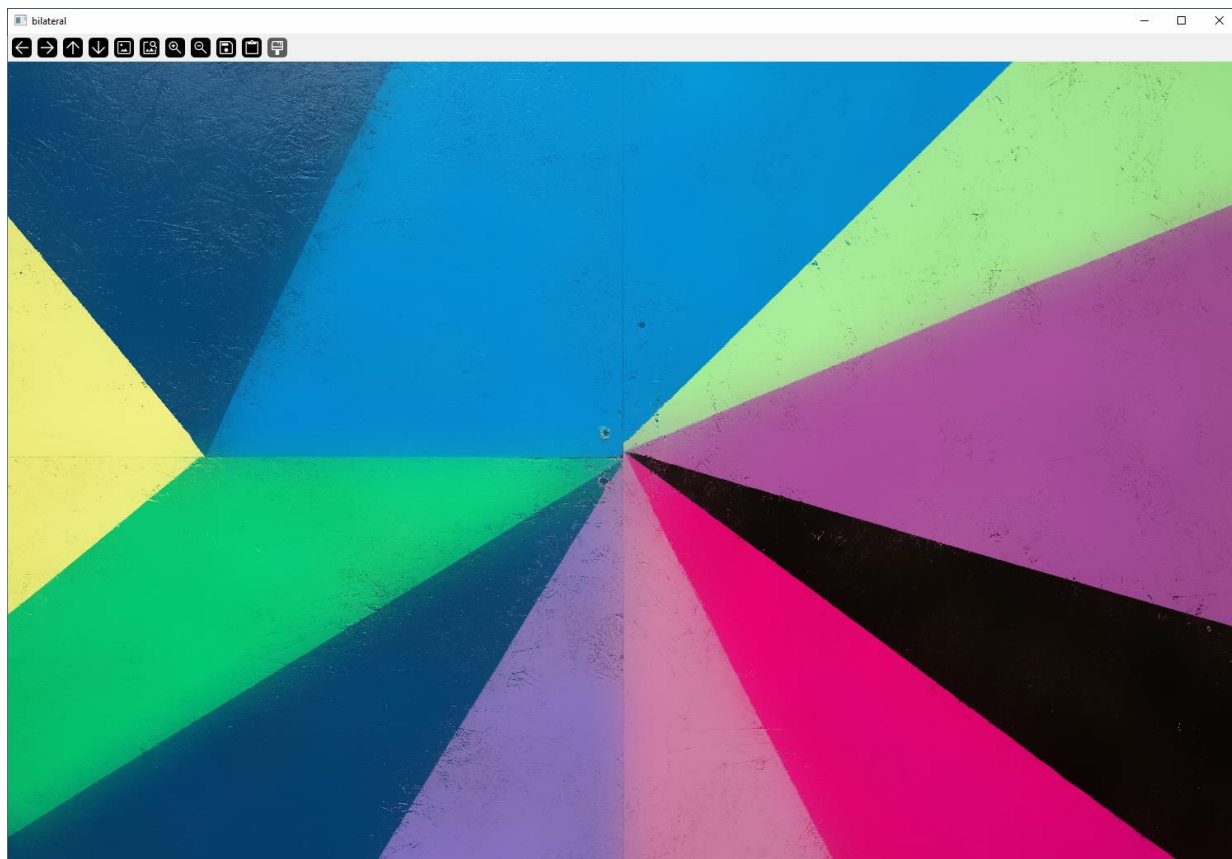


Рисунок 20 – Изображение с применением bilateralFilter

Для того, чтобы максимально убрать все неровности на изображениях лучше всего подходят медианный фильтр и билатеральный фильтр. Медианный фильтр чуть лучше убирает неровности изображения, однако имеет много размытия на границах. Билатеральный фильтр, в свою очередь, решает проблему с размытием границ, однако не так хорошо убирает неровности изображения.

Задание 6:

Листинг 6:

```
import cv2
import numpy as np

# for mac
# path5 = "/Users/new/Desktop/code/cv/lab1/1_5.jpg"
# for win
path5 = r"C:\Users\user\Desktop\course4labs\cv\lab1\1_5.jpg"
```

```
image = cv2.imread(path5, cv2.IMREAD_COLOR)
median = cv2.medianBlur(image, 63)
```

```
cv2.imshow("original", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
cv2.imshow("median", median)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

На рисунках 21-22 изображены картинки, полученные в результате работы программы.

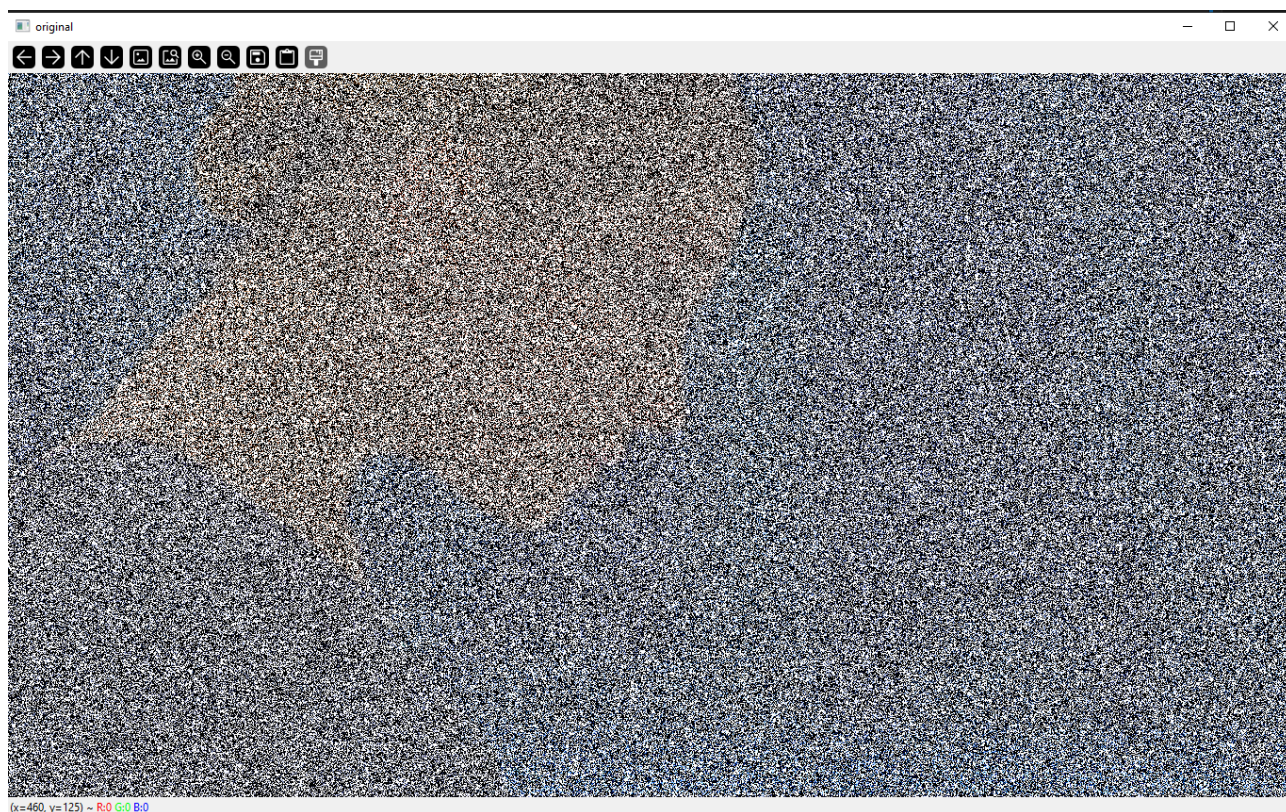


Рисунок 21 – Изображение с шумом

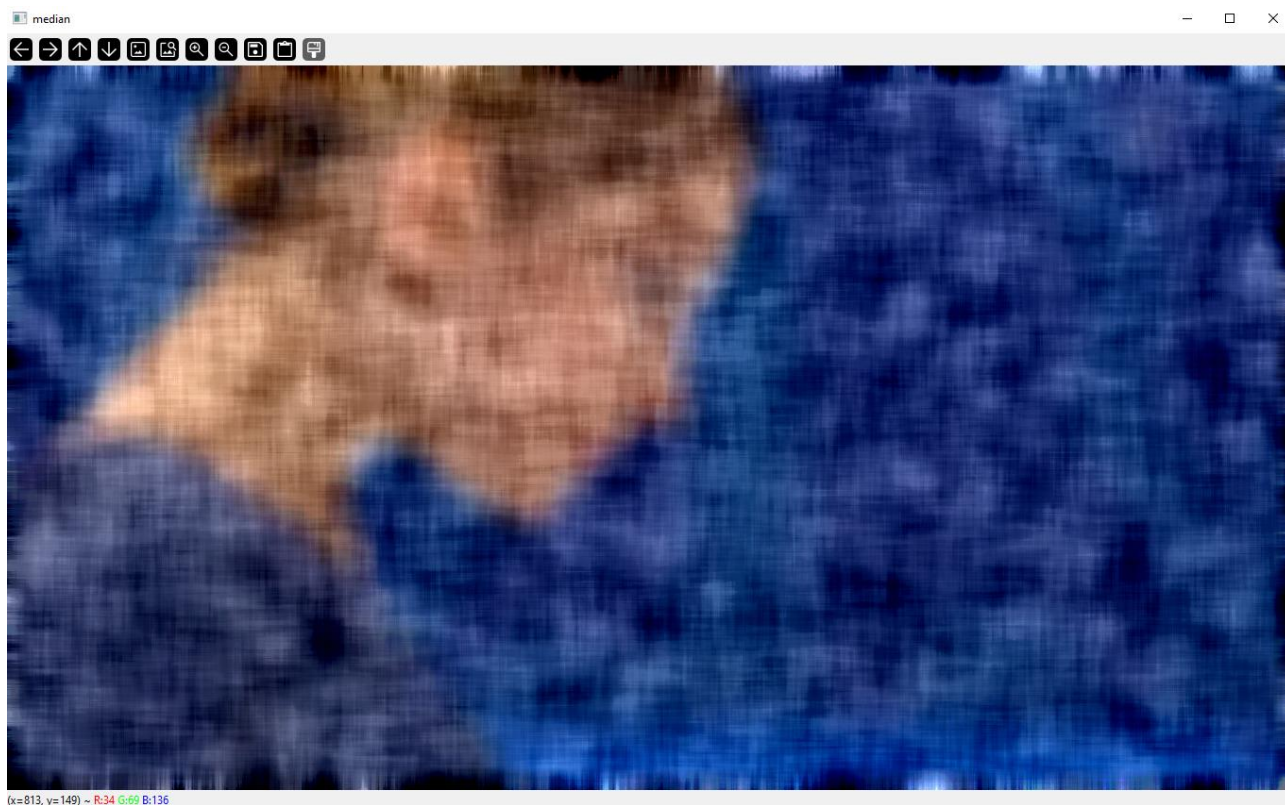


Рисунок 21 – Изображение с применением медианного фильтра

В результате применения медианного фильтра изображение было избавлено от шума и можно разглядеть очертания предмета на картинке.

Вывод: в результате лабораторной работы были освоены основные операции с изображениями, методы фильтрации изображений, пороговые преобразования.